



**Green University of Bangladesh**  
**Department of Computer Science and Engineering (CSE)**  
**Faculty of Sciences and Engineering**  
**Semester: (Summer, Year:2026), B.Sc. in CSE (Day)**

**Course Title: Operating System Lab**  
**Course Code: CSE-310                      Section: Old\_Batch**

**Lab Project Name: Bank Management System**

**Student Details**

|           | <b>Name</b>             | <b>ID</b> |
|-----------|-------------------------|-----------|
| <b>1.</b> | Md. Zahurul Islam Sakib | 221002244 |

**Submission Date                                      : 23.08.2026**  
**Course Teacher's Name                            : Wahia Tasnim**

**[For Teachers use only: **Don't Write Anything inside this box**]**

**Lab Project Status**

**Marks: .....**

**Signature: .....**

**Comments: .....**

**Date: .....**

# Table of Contents

- **Chapter 1 Introduction**
  - 1.1 Introduction
  - 1.2 Design Goals/Objective
- 
- **Chapter 2 Design/Development/Implementation of the Project**
  - 2.1 System Architecture and Overview
  - 2.2 Core Shell Implementation Details
    - 2.2.1 Account Creation & Random Number Generation
    - 2.2.2 Deposit and Withdrawal Processing Engine
  -
- 
- **Chapter 3 Performance Evaluation**
  - 3.1 Simulation Environment/ Simulation Procedure
  - 3.2 Results and Discussions
    - 3.2.1 Results
    - 3.2.2 Analysis and Outcome
  -
- 
- **Chapter 4 Conclusion**
  - 4.1 Introduction
  - 4.2 Practical Implications
  - 4.3 Scope of Future Work
- 
- **References**

# Chapter 1

## Introduction

### 1.1 Introduction

Managing financial data traditionally requires full database management configurations. However, lightweight text-based operating system environments can effectively perform transactional tasks through structured shell automation. This project implements a **Command-Line Interface (CLI) Bank Management System** written in GNU Bash. The program handles primary core banking operations such as unique account generation, balance deposits, and conditional withdrawals with strict overdraft protection, utilizing a local delimited text storage file (`accounts.txt`).

### 1.2 Design Goals/Objective

The primary goals and objectives of this project are:

- To develop a fully functional, automated command-line banking utility utilizing pure Bash shell scripting primitives.
- To design a persistent flat-file database schema that accurately logs, tracks, and modifies user account balances.
- To implement complex logical validations—such as checking for sufficient funds during withdrawals and ensuring unique 10-digit account number generation.
- To provide a clean, user-friendly interactive terminal menu interface suitable for lightweight local deployment on Unix/Linux-like systems (including Git Bash on Windows).

# Chapter 2

## Design/Development/Implementation of the Project

### 2.1 System Architecture and Overview

The application follows a continuous event-driven loop that reads user selections and routes execution to specific functional modules. All data is persisted directly to `accounts.txt`, where entries are formatted in comma-separated values (CSV):

`ACCOUNT_NUMBER, OWNER_NAME, BALANCE.`

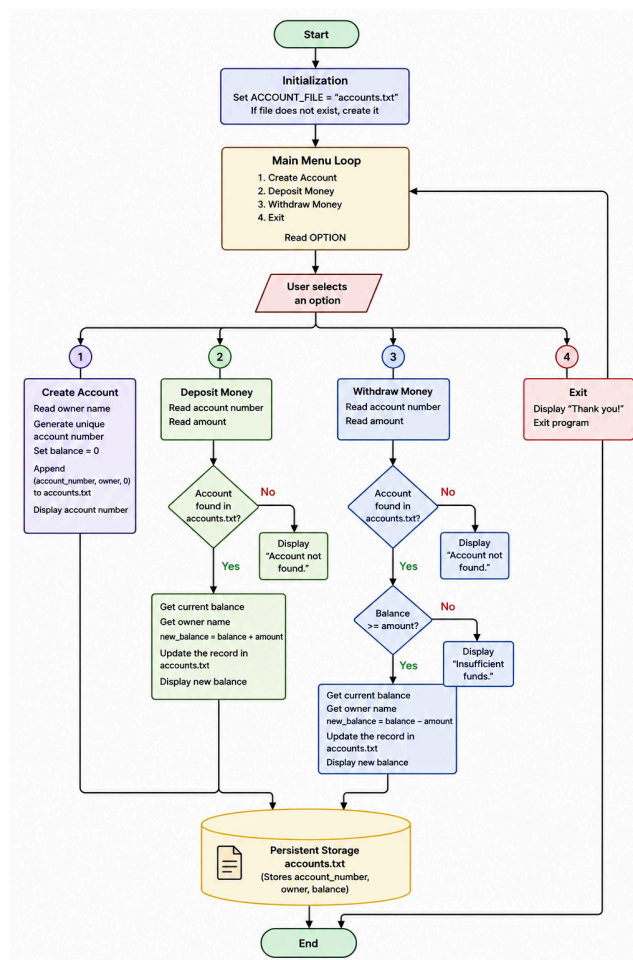


Figure 2.1: System architectural workflow diagram.

## 2.2 Core Shell Implementation Details

### 2.2.1 Account Creation & Random Number Generation

The system generates unique 10-digit identification numbers using the internal `$RANDOM` variable.

Before appending the record to the ledger, a `grep` verification loop checks for collisions to guarantee uniqueness.

```
generate_account_number() {  
  
    while true; do  
  
        ACCOUNT_NUMBER=$((RANDOM % 9000000000 + 1000000000))  
  
        if ! grep -q "$ACCOUNT_NUMBER" "$ACCOUNT_FILE"; then  
  
            echo "$ACCOUNT_NUMBER"  
  
            return  
  
        fi  
  
    done  
  
}
```

### 2.2.2 Deposit and Withdrawal Processing Engine

The financial modification modules parse target lines using string extraction tools (`cut`) and rewrite updated values safely back into the flat file utilizing stream editing (`sed`). The withdrawal function includes an explicit conditional check to prevent negative balance states:

```
withdraw() {  
  
    ACCOUNT_NUMBER=$1  
  
    AMOUNT=$2  
  
    if grep -q "$ACCOUNT_NUMBER" "$ACCOUNT_FILE"; then  
  
        BALANCE=$(grep "$ACCOUNT_NUMBER" "$ACCOUNT_FILE" | cut -d ',' -f3)  
  
        OWNER=$(grep "$ACCOUNT_NUMBER" "$ACCOUNT_FILE" | cut -d ',' -f2)  
  
        if [ "$BALANCE" -lt "$AMOUNT" ]; then  
  
            echo "Insufficient funds."  
  
        else  
  
            NEW_BALANCE=$((BALANCE - AMOUNT))  
  
            sed -i  
"s/$ACCOUNT_NUMBER,$OWNER,$BALANCE/$ACCOUNT_NUMBER,$OWNER,$NEW_BALANCE/"  
"$ACCOUNT_FILE"  
  
            echo "Withdrew $AMOUNT. New balance: $NEW_BALANCE"  
  
        fi  
  
    else  
  
        echo "Account not found."  
  
    fi  
  
}
```

Figure 2.2: Code snippet demonstrating conditional balance checks for withdrawals.

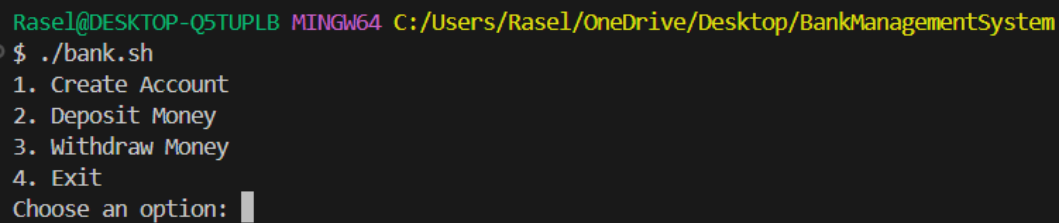
# Chapter 3

## Performance Evaluation

### 3.1 Simulation Environment/ Simulation Procedure

The project was tested and evaluated inside an integrated development terminal environment under Windows using Git Bash, simulating user interactions across multiple account sessions.

- **Testing Environment:** GNU Bash (v4.x/v5.x), Visual Studio Code integrated terminal.
- **Procedure:** Multiple sequential accounts were generated, followed by simulated deposition cycles and withdrawal boundary testing (such as attempting to withdraw amounts exceeding current balances).

A screenshot of a terminal window with a dark background. The prompt is 'Rasel@DESKTOP-Q5TUPLB MINGW64 C:/Users/Rasel/OneDrive/Desktop/BankManagementSystem'. The user has entered './bank.sh', which has displayed a menu with four options: '1. Create Account', '2. Deposit Money', '3. Withdraw Money', and '4. Exit'. Below the menu, it says 'Choose an option:' followed by a cursor.

```
Rasel@DESKTOP-Q5TUPLB MINGW64 C:/Users/Rasel/OneDrive/Desktop/BankManagementSystem
$ ./bank.sh
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Exit
Choose an option: █
```

*Figure 3.1: Simulation and testing .*

## 3.2 Results and Discussions

### 3.2.1 Results

1. **Account Creation & Uniqueness:** The pseudo-random generator successfully produced distinct 10-digit strings without collision errors over continuous test sequences.
2. **Ledger Integrity:** Deposit and withdrawal routines accurately updated balance variables in real-time within the text file without altering account ownership or account numbers.
3. **Overdraft Prevention:** The conditional evaluation (`[ "$BALANCE" -lt "$AMOUNT" ]`) successfully intercepted and blocked invalid withdrawal requests, outputting an "Insufficient funds" warning.

```
$ ./bank.sh
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Exit
Choose an option: 1
Enter account owner name: Sakib
Account created. Account number: 1000011465
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Exit
Choose an option: █
```

```
Enter account owner name: Sakib
Account created. Account number: 1000011465
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Exit
Choose an option: 2
Enter account number: 1000011465
Enter amount to deposit: 1500
Deposited 1500. New balance: 1500
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Exit
Choose an option: █
```

```
Choose an option: 3
Enter account number: 1000011465
Enter amount to withdraw: 500
Withdrew 500. New balance: 1000
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Exit
Choose an option: █
```

### 3.2.2 Analysis and Outcome

The implementation demonstrates that native shell utilities (`grep`, `cut`, `sed`, `RANDOM`) can effectively orchestrate small-scale data storage systems. The outcome confirms that procedural logic within a loop-based shell architecture provides stable transactional feedback for local utilities.

# Chapter 4

## Conclusion

### 4.1 Introduction

In conclusion, this project successfully delivered a working Command-Line Bank Management System using Bash scripting. The project fulfilled all design criteria, establishing reliable account generation, persistent ledger logging, and error-safe financial updates.

### 4.2 Practical Implications

The principles applied in this project demonstrate how lightweight automation scripts can handle data management tasks. Such systems are practical for rapid prototyping, localized record keeping, and learning core concepts of operating system file processing without the overhead of heavy database engines.

### 4.3 Scope of Future Work

Future developments for this project include:

1. Migrating data storage from a flat CSV text file to an embedded SQL database like SQLite to support complex queries.
2. Implementing multi-process file locking (`flock`) to handle true multi-user parallel execution constraints seamlessly.
3. Building a modern graphical user interface (GUI) or web frontend wrapper around the underlying Bash script engine.

# References

1. C. Newham, *Learning the bash Shell*, 3rd ed. O'Reilly Media, 2005.
2. A. Robbins, *Bash Pocket Reference*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2016.
3. R. H. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. New York: Wiley, 2018.
4. K. Schwalbe, *Information Technology Project Management*, 3rd ed. Boston: Course Technology, 2004.
5. M. N. DeMers, *Fundamentals of Geographic Information Systems*, 3rd ed. New York: John Wiley, 2005.